CRYPTOGRAPHY · COURSE PROJECT · GROUP 18

Secure Cloud File Sharing Using Hybrid Cryptography



We rebuilt the Hybrid AES-ECC model of Rehman et al. (Electronics, 2021), attacked it, and fixed it with authenticated hybrid encryption.

RESEARCH + APPLICATION PROJECT

Saurav Singh
S20230010219

Rohan Dubey
S20230010207

Ujjawal
S20230010245

Aditya Raj
S20230030364

Live demo: CryptoVault (MongoDB · Express · React · Node.js). Every number in this deck was measured by us.

Why cloud files need hybrid cryptography

PRESENTER Saurav Singh

THE SETTING

Cloud storage means our files live on someone else's computer. Hybrid cryptography uses two kinds of cryptography together:

AES
symmetric and fast: locks the file

ECC
public key: protects and shares the AES key

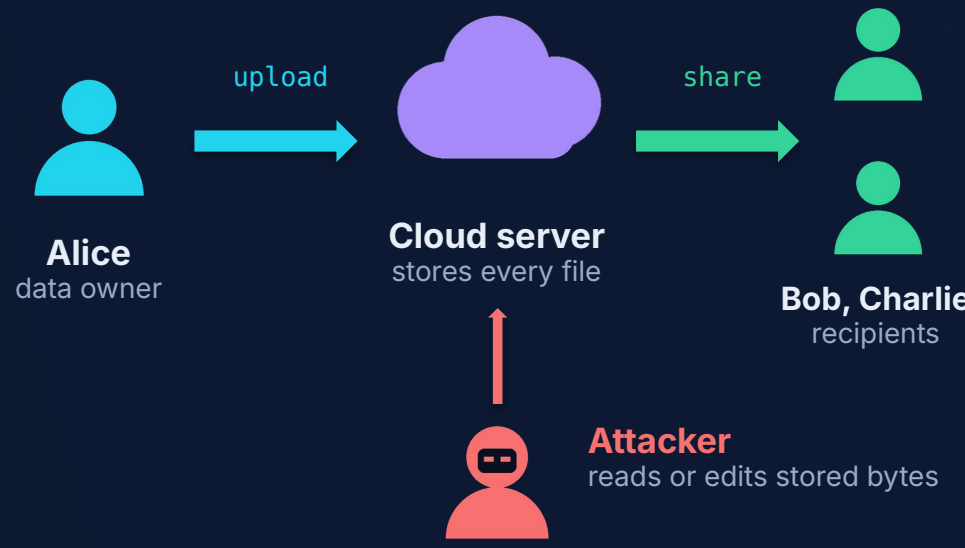
WHO CAN ATTACK THE FILES?

Outsider with a leaked copy
misconfigured bucket, stolen backup

Curious or malicious cloud insider
can read and change the stored bytes

A user whose access was removed
must not open newer versions

OUR THREAT MODEL



PROBLEM STATEMENT

Does the published Hybrid AES-ECC model (Rehman et al., 2021) really keep shared cloud files secret and unmodified? If not, can we fix it without making it slow?

1 Rebuild | 2 Attack | 3 Fix | 4 Measure

What the base paper uses: AES + ECC

PRESENTER Rohan Dubey

AES Module 3

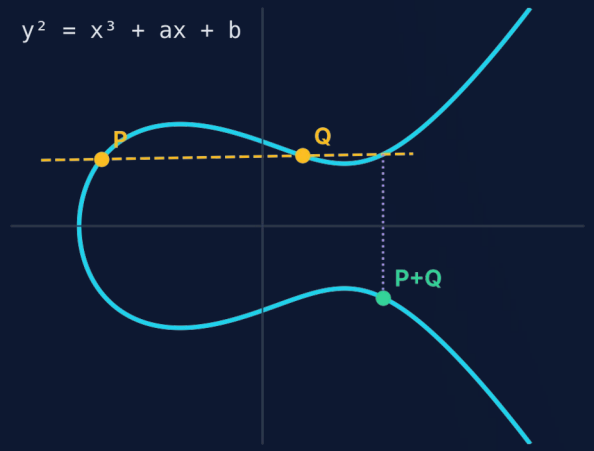
Symmetric block cipher: one secret key locks and unlocks. 128-bit blocks, 10 rounds for a 128-bit key.

- **SubBytes** S-box substitution
- **ShiftRows** rotate rows
- **MixColumns** math in GF(2^8)
- **AddRoundKey** XOR round key

× 10 rounds

Very fast, so it encrypts the file itself.

ECC Module 4



Private key: a secret number **n**
Public key: the point **P = n·G**
Getting n back from P (the ECDLP) is infeasible.
256-bit ECC ≈ 3072-bit RSA security.

Hybrid AES-ECC

the paper's algorithm (Box 1 + Box 2)

1. **Cloud generates ECC keys**
   P = n·G
2. **AES key taken from the**
   ECC public key P
3. **AES encrypts the file**
   at the cloud provider
4. **Cloud stores ciphertext**
   decrypt = reverse steps

Claims: smaller keys, faster, and "authentication & integrity"

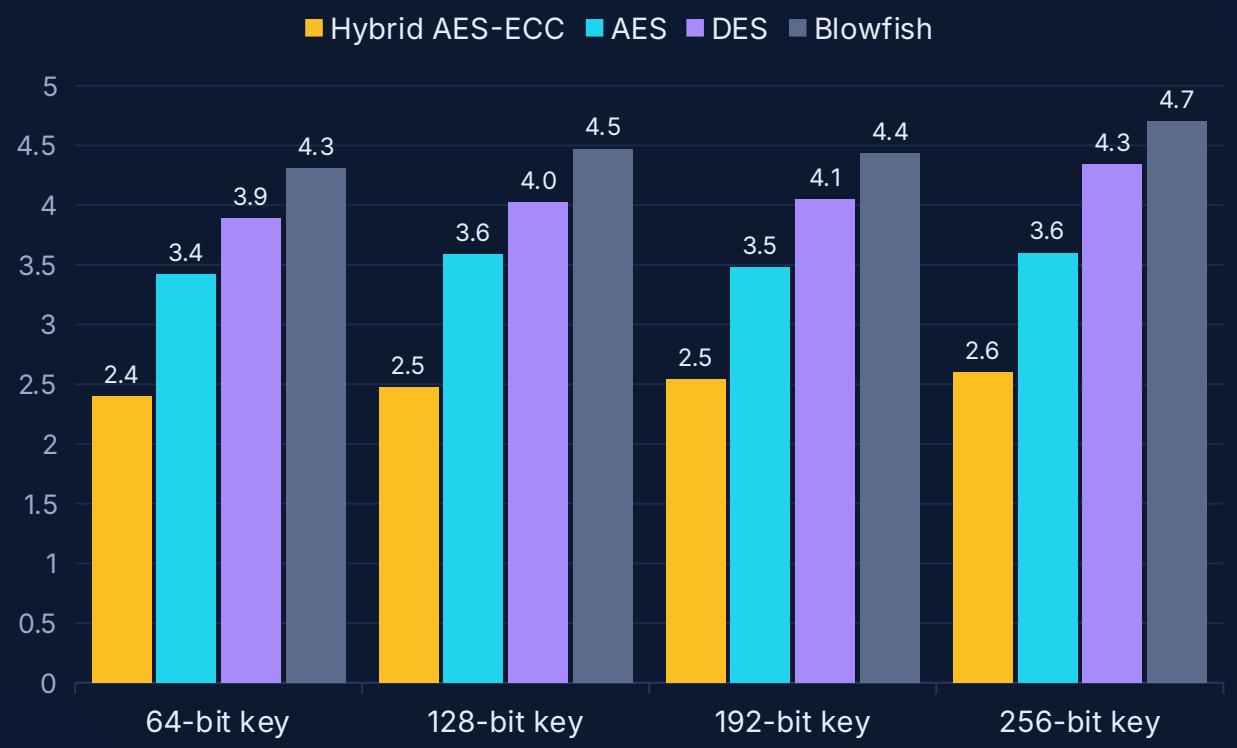
OUR RE-IMPLEMENTATION

We followed the paper's Box 1 and Box 2 exactly. The authors' GitHub repository contains only a README, so we wrote the code ourselves: MongoDB · Express · React · Node.js, NIST P-256 curve, AES-128.

What the paper reports, and our reproduction

PRESENTER Rohan Dubey

ENCRYPTION TIME REPORTED IN THE PAPER (S)



Source: Rehman et al., Electronics 10(21):2673, Table 7 (text data, Python, Intel i7-8550U)

WHAT THE PAPER CLAIMS

~30% faster
Hybrid 2.40–2.60 s vs AES 3.42–3.60 s (Table 7)

61% / 64%
avalanche for 1-bit key / text change; AES: 51% / 55% (Table 9)

1 : 12
key size ECC : RSA, so less memory (Table 5)

"Integrity"
says it ensures authentication & data integrity (Abstract)

OUR REPRODUCTION

Works ✓
100/100 test files encrypt and decrypt correctly

928 MB/s
AES-128-ECB speed on a 16 MiB file

0.28 ms
P-256 key generation (RSA-3072: 145 ms)

It is fast. But is it secure?

Three challenges we found and proved

PRESENTER Ujjawal

1 The key comes from the PUBLIC key

Box 2 builds the AES key from the ECC public key. Public keys are public, so anyone can rebuild the AES key without the private key.

100 / 100
files decrypted by our attacker using only public data

Kerckhoffs' principle · Shannon: key equivocation H(K | P) = 0

2 No integrity check, and ECB mode

The paper claims integrity but adds no MAC, hash or signature. The cloud can change data silently, and ECB keeps the patterns of the data.

0 / 1,250
tampering attempts detected (bit flips, block swaps, deletions)

Block cipher modes (Module 3) · Hash & signatures (Module 5)

3 No real sharing or revocation

The cloud does the encryption, so it sees the plaintext and can compute the key. Removing a user changes no key at all, so revocation is only on paper.

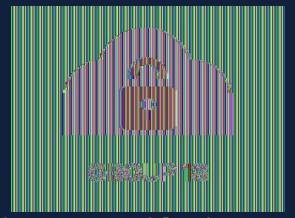
0 keys
change when a user's access is revoked

Key distribution & management (Unit 6)

EVIDENCE: SAME IMAGE, ENCRYPTED



original


base: 98% blocks repeat


ours: 0% repeat

EVIDENCE: CLOUD SWAPS 2 CIPHERTEXT BLOCKS

TO: ALICE	AMT: Rs 100000	→	**Rs 10**
TO: MALLORY	AMT: Rs 10	→	**Rs 100000**

Decrypts with no error. Nobody notices.

Our fix: an authenticated hybrid envelope

PRESENTER Aditya Raj

1 — Random file key
256-bit key from the browser's secure random generator, new for every file
FIXES 1

2 — AES-256-GCM
encrypts and adds a 128-bit tag (CTR mode + GHASH)
FIXES 2

3 — Lock key per user
ECDH (Diffie-Hellman on P-256) + HKDF wraps the file key
FIXES 1 · 3

4 — Owner signs
ECDSA-P256 over SHA-256 of everything uploaded
FIXES 2

5 — Cloud sees nothing
stores only ciphertext: never the plaintext, keys or file names
FIXES 3

WHEN A RECIPIENT OPENS THE FILE

Verify owner's ECDSA signature → Check SHA-256 of ciphertext → Unlock file key with MY private key → Decrypt + check GCM tag

Any check fails → **BLOCKED**. No data is released.

SHARING AND REVOKING

Share: add one more locked key (2.7 ms), no re-encryption.

Revoke: new random key + re-encrypt as version 2 (7.6 ms per MiB). The old key opens nothing new.

BUILT ONLY FROM COURSE TOPICS

Module 3
AES · modes: ECB × → CTR/GCM ✓

Units 4 & 6
ECC · Diffie-Hellman key exchange

Module 5
SHA-256 · digital signatures (DSA idea on curves)

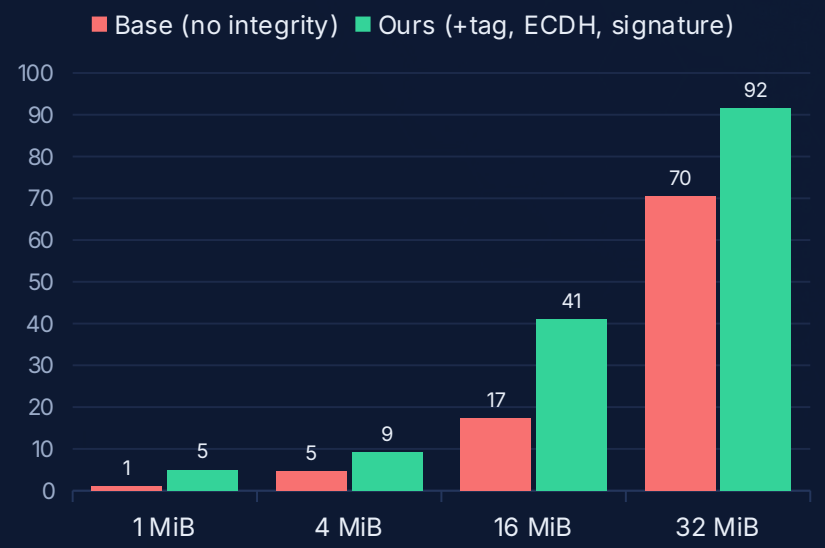
Unit 2
Entropy: key = 256 random bits; ciphertext ≈ 8 bits/byte

Results after the fix, and conclusion

PRESENTER Aditya · Saurav

Test (all measured in our app)	BASE AES-ECC	OURS
Decrypt using only public data (100 files)	100 / 100	0 / 100
Tampering detected (1,250 attacks)	0 %	100 %
Image patterns leaked (repeated blocks)	98.2 %	0 %
Ciphertext entropy (max 8 bits/byte)	6.55	8.00
Same file uploaded twice: bits that differ	0 %	49.8 %
Revoke a user	not possible	7.6 ms / MiB
Encrypt a 16 MiB file	17.2 ms	41.0 ms

ENCRYPTION TIME (MS)



Cost of security: about 3 ms per file plus extra time on big files. Still 390 MB/s.

CONCLUSION

- The published AES-ECC model is fast but not secure: anyone can compute its AES key from public data, and tampering goes unnoticed.
- Our fix uses only course tools (AES-GCM, ECDH, ECDSA, SHA-256). All our attacks fail, and every tampering attempt is caught.
- Security costs a few milliseconds per file. Future work: post-quantum key exchange (ML-KEM) and a public-key transparency log.